CS 530 Project Proposal

Tzung-ying Hsieh*
hsieh65@purdue.edu
Purdue University
West Lafayette, Indiana, USA

Tejas Yalamanchili†
tyalaman@purdue.edu
Purdue University
West Lafayette, Indiana, USA

Keywords

Fluid Dynamics, Scientific Visualization

ACM Reference Format:
Tzung-ying Hsieh and Tejas Yalamanchili. 2026. CS 530 Project Proposal. In . ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Rivers and steep creeks often exhibit complex hydraulic behaviors caused by the interaction between flowing water and the surrounding river beds. When water encounters various obstacles such as submerged rocks, logs, uneven river beds , it can form distinctive flow patterns including eddies recirculating holes and turbulent waves. These hydraulic features are not only important for understanding natural water dynamics but can also pose safety risks, as strong recirculating flows can trap people or object in place. Our project focuses on developing a visualization tool that illustrates how such hydraulic features form and evolve as the physical environment changes. By allowing users, to modify terrain elements such as rocks or logs within the stream, the system will demonstrate how these obstacles will alter water flow and generate different hydraulic structures. The simulation data that we plan to use will be generated using Taichi for fluid modeling, while VTK will be used to visualize the resulting flow patterns and terrain features.

2 Data

The dataset is generated entirely using Taichi, a Python package. Obstacles are introduced by raising values in the bed elevation input array b. The dataset will be generated using parameters that represents a generic stream.

2.1 Data Set

The Taichi parameters are described on the table at 1. It defines the domain and obstacles. Obstacles are introduced by raising values in the bed elevation input array. Rocks are bumps from the river bed and logs are ridges. Solid boundaries are cells where the input array $b \geq h_0$ this will force water around it. The simulation dataset will span 300 frames.

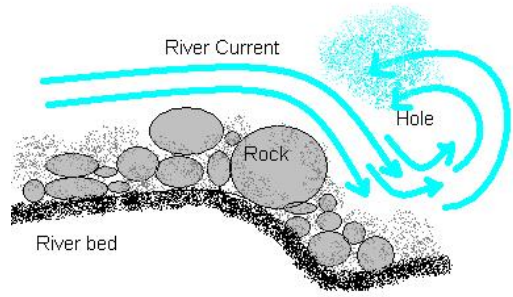

Figure 1: Diagram of a Hole

2.2 Conversions

After each frame is exported from Taichi, it contains six field arrays. The field contains, height h, velocity v_x, v_y, speed $|u|$, vorticity ω, and pressure p. Using `numpy_to_vtk()`, the objects will be converted to a vti file so vtk can be read into `vtkImageData`.

2.3 Obstacle Dataset

Since Taichi does not contain direct obstacle output, they are constructed procedurally in VTK using `vtkPolyData`. To guarantee uniformity of obstacles between inputs to Taichi and VTK visualization, a set of possible obstacles and its configurations will be precomputed. Visual trial and error will produce a general approximation of the rocks and logs. First, we will create a `vtkSphereSource` and noise will be added to the surface of the rock to create a rock like texture when visualized. For log, `vtkCylinderSource` will be used. Since we preconfigure possible rocks and logs, using the diameter of the spheres, we now know the possible parameters to provide Taichi's river bed array.

3 Methodology

The VTK pipeline has two independent inputs that converge in a single `vtkRenderer`. The first is the `.vti` time-series from Taichi, which drives the fluid visualization. The second is obstacle geometry from precompute geometries for VTK. Taichi encodes obstacles only as elevated values in the bed array b and produces no geometry objects. Instead, rock and log meshes are constructed procedurally at render time using `vtkSphereSource` and `vtkCylinderSource`.

Listing 1: Fluid pipeline

```
vtkXMLImageDataReader -> vtkWarpScalar
  -> vtkPolyDataMapper -> vtkActor
```

*Both authors contributed equally to this research.
†Both authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Table 1: Taichi Simulation Inputs

Parameter	Type / Shape	Value	Description
Domain resolution	`int [x, y]`	(512, 256)	x and y dimension of grid.
dx, dy	`float , float`	$dx = 0.039\,\mathrm{m}$, $dy = 0.039\,\mathrm{m}$	Cell size representing size per cell in grid.
h_0	`float32[N_x,N_y]`	$0.3 - 0.7\,\mathrm{m}$	Surface height at (x, y), representing depth of stream.
u_0	`float32[N_x,N_y,2]`	$u_x \approx 0.5\,\mathrm{m/s}$, $u_y = 0\,\mathrm{m/s}$	Initial velocity per cell, each element in (u_x, u_y) represents velocity in x and y direction.
b	`float32[N_x,N_y]`	rocks $\approx 0.3\,\mathrm{m}$, logs $\approx 0.5\,\mathrm{m}$, riverbed $= 0\,\mathrm{m}$	Bed elevation encodes obstacles at cell (x, y). $b_{(x,y)} > 0$ represents rocks/logs.
ν	`float`	$0.001\,\mathrm{m}^2/\mathrm{s}$	Kinematic viscosity, how "thick" or "sluggish" the fluid behaves when it moves.
g	`float`	$9.81\,\mathrm{m/s}^2$	Gravitational acceleration constant.
Δt	`float`	0.0088 s	The time difference between each solver step.
Export interval	`int`	0.1 s	How frequent each frame is outputted.
Simulation time	`float`	20 s	200 frames $\times$ 0.1 s.

Listing 2: Obstacle pipeline

```
vtkSphereSource / vtkCylinderSource
  -> vtkPolyDataMapper -> vtkActor
```

3.1 VTK Pipeline

The following pipeline in VTK will be used for rendering and interactivity:

- **3D Surface Height** — `vtkWarpScalar` is used to create the 3D surface of the waves. Height field $h(x, y)$ of the water will show the crests and troughs at each location.A color map will show the flow speed.
- **Velocity Representation** — Arrows will represent the flow direction and speed at that point in the stream. This will reveal recirculation zones and stagnation points. `vtkGlyph3D` will use used to scale by speed and orient with the flow.
- **Recirculation Representation** — Recirculation features such as holes or eddies will be represented by iso contours. Contour lines connecting all points with the same rotational intensity will represent a vorticity level, with each recirculating area containing multiple. `vtkContourFilter` will be used to achieve such visualization.
- **Overall Trajectory** — Streamlines will represent the overall trajectory of the water flow. Streamlines will be seeded along the inlet. They show the overall topology of the flow, where the flow goes around a rock or speeds up in narrow sections. We employ `vtkStreamTracer` for this purpose.
- **Obstacle Rendering** — Rocks are built using `vtkSphereSource` with radius matching its respective bump in b. Log meshes use `vtkCylinderSource` with length, radius, and orientation matching the ridge in b.`vtkTransform` is used position both obstacles.

3.2 Interactivity

The visualization will support time animation. The animation can be by step or autoplay through 300 frames. The user may toggle between visualizing different scalar fields: stream height, stream speed and stream vorticity (how much the flow is spinning). The application shall allow inserting preconfigured obstacles into the stream and recomputation of flow. The specific GUI design and use case example will be in the next sections.

4 GUI

The graphical interface is implemented in PyQt6 as a side panel docked alongside the VTK render window with a slider panel beneath. The sidebar is divided into three functional panels: Display, Obstacle Editor, and Simulation Parameters, communicating with VTK and Taichi through a shared application state object. The bottom slider panel will have one panel to control the playing of the animation.

4.1 Render Window

The render window will contain the rendered visualization. It will display the results according to what the user provided and simulation outcome. Additionally, it will provide a colorbar for visualizations which contain magnitude. Lastly, a "coordinate system" with x and y value of the current mouse location will be displayed in a corner to help users select locations to insert obstacles. `vtkCornerAnnotation` or `vtkActor` are used to display the coordinate while `vtkPropPicker` will be used to find the mouse's x and y coordinate.

4.2 Bottom Slider Panel

A `QTimer` will control the advances of the frames.

- **Play / Pause (`QPushButton`)** — Toggles the `QTimer` and advances or pauses the frame.
- **Step Forward / Back (`QPushButton`)** — Increment or decrement the frame index for frame-by-frame inspection.
- **Frame slider (`QSlider`)** — Spans frames 0–300. A `QLabel` displays the current frame number and simulation time (frame $\times$ 0.1 s).

4.3 Sidebar Panel

The first section of the sidebar panel will control the visualization options.

- **Scalar field selector (`QComboBox`)** — This selector allows the use to select height, speed, or vorticity.

- **Layer toggles (`QCheckBox` × 4)** — Control visibility of the water surface, velocity glyphs, vorticity contours, and streamlines via `VisibilityOn()` / `VisibilityOff()`.

The second section of the sidebar panel will control the obstacles.

- **Obstacles Display (`QTreeWidget`)** — This will display the current obstacles on the screen. It will have columns that show the type of preconfigured obstacle (e.g. log-1), X location, Y location. Double-clicking a row opens a `QDialog` modal with a `QPushButton` button which deletes the selected row.
- **Obstacle Selector `QVBoxLayout`** Each box will contain a preconfigured obstacle. A `QLabel` will show the obstacle's name and size. Two `QLineEdit` fields will allow the user to enter the intended X and Y position of the obstacle. Lastly, a `QPushButton` will allow the user to add the obstacle.
- **Re render Button `QPushButton`** This button launches a `QThread` worker that rebuilds b via `b.from_numpy()`, runs the SWE solver, and writes new `.vti` files.
- **Information Bar `QStackedWidget`** Provides any information of the program. It may be a a `QProgressBar` receives progress updates via Qt signals emitted by the worker. During re-simulation controls are disabled via `setEnabled(False)` and re-enabled on the worker's `finished` signal. When a user attempts to insert an obstacle at a illegal position, a `QLabel` with error text will appear.

5 Use Case Scenario

In an example, the user will first open the application which will produce a stream without obstacles in it. The user can then click on the buttons to add preconfigured logs or rocks as obstacles at the positions they entered in (x, y) coordinate the. The application will by default place the obstacle on the riverbed with $z = 0$.

Upon placing the obstacles, the user will request the application to re render. The application will load for the time required for the fluid simulation to compute. Upon completion of the simulation, the application window will by default show the stream height at the starting frame $t = 0$. The user can toggle between showing height, speed, or vorticity.

The user may now also edit the obstacle. They can select the obstacles that exists in the sidebar and use a button to remove them. They can insert new obstacles as detailed previously. A re render must be requested for new obstacles. Once the re render is completed, the user can use the bottom bar to start the simulation. Upon each re render, the time bar gets reset to $t = 0$. The simulation will run for 200 frames. The user can also move between specific frames using a slider bar. It is also possible to toggle between height, speed, or vorticity.

6 Milestone

- Week 1: Produce preconfigured obstacles.
- Week 2: Taichi SWE solver implementation. We will work on generating sample datasets with Taichi using the preconfigured obstacles.
- Week 3: Work on VTK pipeline with surface warp, colormap, glyphs able to produce rendered height field with velocity arrows
- Week 4: Work on multilayer visualization woth vorticity contours, streamlines, and obstacles
- Week 5: Validation of results visually and incorporate interactivity with user selection of obstacles or visualization layer
- Week 6: Write-up, complete interactive application, bug fixes, and final report